Viva Quick Guide

Real-Time Database Monitor & Alert System · read this the night before

Short and simple. Every page is about the **rubric**: which file earns which mark, what it does in plain words, and what to say. No code: open the file in the editor if sir asks to see it.

The rubric tags used in this guide

Design Inheritance Collections Exceptions JDBC GUI Threads Sockets

Marks: Design 1 · Inheritance 1 · Collections 1 · Exceptions 1 · JDBC CRUD 2 · GUI 1 · Threads 1 · Viva 1. Sockets aren't a separate rubric line; they support Design and Threads.

What's inside

1. The project in 30 seconds
2. Rubric → files (the one table to remember)
3. Every file, grouped, with its rubric tags
4. Each rubric line: simple words, where, what to say
5. Demo order and who says what
6. Quick questions and answers

Code on GitHub `main` (20 Oct 2026). Diagrams are in `docs/diagrams/`: System_Architecture, UML_Class_Diagram, ER_Diagram.

1. The project in 30 seconds

THE STORY

A shop has **CCTV**. Anyone can change a price tag: a staff member at the counter, or someone sneaking in the back. The CCTV (**MySQL triggers**) records every change. A **security guard** (our server) checks the recordings every 2 seconds with a **rule book** ("price cut in half = serious"). When a rule is broken, he writes it in the **incident register** (alerts table) and announces it on the **walkie-talkies** (dashboards). The **manager** (Admin console) marks incidents as handled.

Program	Start with	In the story	Rubric
Server	`run server`	Security guard	Threads · Sockets · JDBC
Dashboard	`run client`	Walkie-talkie	GUI · Sockets · Collections
Admin console	`run admin`	Manager's office	GUI · JDBC · Collections
Employee app	`run employee`	Billing counter	GUI · JDBC · Exceptions
MySQL	`sql/setup.sql`	Shop + CCTV recorder	JDBC

THE PITCH (MEMORISE)

We watch a database table for risky changes from anywhere. MySQL triggers record each change, our server checks it against 10 rules, and alerts are pushed to live dashboards over sockets in 2–4 seconds. Three Swing apps: a dashboard, an Admin console and an Employee app.

2. Rubric → files

If you remember one page, remember this one.

Rubric	Main files	One line to say
Design 1 mark	3 diagrams in `docs/diagrams`; the 7 packages	Each package has one job; screens never touch SQL directly.
Inheritance 1 mark	**`Alert`** + 3 kinds; **`Rule`** + 7 kinds; **`AlertCard`**	Abstract parent, children fill in the blanks; the dashboard never checks the type.
Collections 1 mark	**`DashboardFrame`**, **`BroadcastServer`**, **`ClientHandler`**, **`AlertTableModel`**	HashMap = no duplicate cards; queue per dashboard; sort with Comparator.
Exceptions 1 mark	**`InvalidAlertException`**, **`InvalidProductException`**, **`ChangeDAO`**, **`AlertListenerThread`**	Own checked exceptions; rollback; nothing crashes, everything retries.
JDBC 2 marks	**`ProductDAO`**, **`AlertDAO`**, **`ChangeDAO`**, **`ManagedConnection`**, `setup.sql`	Full CRUD with PreparedStatement; one transaction; triggers.
GUI 1 mark	**`AdminFrame`**, **`EmployeeFrame`**, **`DashboardFrame`**	3 Swing apps: tabs, tables, forms, live cards.
Threads 1 mark	**`PollerThread`**, **`ChangeDetectorThread`**, **`BroadcastServer`**, **`DbTask`**	Several workers at once; safe stop; only the EDT touches the screen.
Sockets	**`BroadcastServer`**, **`ClientHandler`**, **`AlertListenerThread`**	Server on port 5050 pushes alert objects; dashboards reconnect.

3. Every file, grouped

All Java files are in `src/dbmonitor/<package>/`. Small helper files are grouped together.

common — shared building blocks, used by every program

File(s)	What it does (plain words)	Rubric
`Alert`	The parent of all alerts. **Abstract**: you can't make a plain alert, only a specific kind. Holds id, severity, message, status. **Read-only** after creation. **Serializable** so it can travel over the network.	Inheritance Sockets Design
`RecordAddedAlert` **`RecordChangedAlert`** **`RecordDeletedAlert`**	The 3 kinds of alert. Each gives its own icon ([ADD] / [CHG] / [DEL]), colour and sentence.	Inheritance
`AlertFactory`	The only place that decides which kind of alert to create (Factory pattern).	Design Exceptions
`Severity`, **`AlertStatus`**	Fixed lists (enums): LOW–CRITICAL with colours; NEW / SENT / RESOLVED.	Design
`InvalidAlertException`	Our own **checked** exception for bad alert data.	Exceptions Inheritance
`AppConfig`	Reads `config/app.properties` (port 5050, 2 s interval, database login).	Exceptions

rules — decides if a change is risky

File(s)	What it does (plain words)	Rubric
`Rule`	Parent of all rules. Checks "is this my table?" the same way for all (**final**), then each child does its own check (**abstract**).	Inheritance
7 rule kinds: **`ThresholdRule`**, **`PercentDropRule`**, **`PercentRiseRule`**, **`DeltaRule`**, **`FieldChangedRule`**, **`RecordInsertedRule`**, **`RecordDeletedRule`**	Each is one type of check, e.g. "price fell by 50%" or "stock below 10".	Inheritance

`ProductRules`	Creates our 10 rules R1–R10 and puts them in one list.	Collections Design
`RuleSet`	Runs all 10 rules on a change; the highest severity wins, so it's one alert per change. List is read-only.	Collections Inheritance
`DataChange`, **`FieldValue`**, **`ChangeType`**, **`DetectionResult`**	Small helpers: one recorded change, its old/new values, and the rules' verdict.	Collections

db — ALL the SQL lives here (DAO = Data Access Object)

File(s)	What it does (plain words)	Rubric
`ManagedConnection`	One database connection that checks itself and reconnects if MySQL restarted.	JDBC Exceptions
`ProductDAO`	**Full CRUD** on products: add, read, update, delete. Checks input first (no negative price).	JDBC Exceptions Threads
`AlertDAO`	Reads alerts, marks them sent, resolves, purges, deletes; writes the broadcast log.	JDBC Collections Threads
`ChangeDAO`	Reads recorded changes (a JOIN). Creates an alert in **one transaction** with rollback.	JDBC Exceptions Collections
`InvalidProductException`	Our own checked exception: "Price cannot be negative".	Exceptions
`Product`, **`BroadcastLogEntry`**	Simple read-only data holders for one row.	Design

server — the background program (security guard)

File(s)	What it does (plain words)	Rubric
`ServerMain`	Starts everything; on Ctrl+C stops the threads neatly (shutdown hook). Clear error if port 5050 is busy.	Threads Exceptions
`ChangeDetectorThread`	Worker #1: every 2 s reads new changes, runs the rules, creates alerts.	Threads Inheritance
`PollerThread`	Worker #2: every 2 s finds unsent alerts, sends them, marks them SENT.	Threads Exceptions

`BroadcastServer`	Opens port 5050, waits for dashboards, keeps their list (CopyOnWriteArrayList), runs a thread pool.	Sockets Threads Collections
`ClientHandler`	One per dashboard: its own queue of alerts, sends each over the socket.	Sockets Threads Collections

client — the live dashboard (walkie-talkie)

File(s)	What it does (plain words)	Rubric
`ClientMain`	Starts the window and the listener.	GUI
`AlertListenerThread`	Connects to the server, receives alerts, shows "Reconnecting" and retries every 3 s.	Threads Sockets Exceptions
`DashboardFrame`	The window: cards in a HashMap by alert id (no duplicates), counters in an EnumMap.	GUI Collections
`AlertCard`	One alert card. Asks the alert for its icon, colour and text, never its type. Turns grey when resolved.	GUI Inheritance

admin — the manager's console

File(s)	What it does (plain words)	Rubric
`AdminMain`, **`AdminFrame`**	The window with 4 tabs; refreshes every 3 s with a Timer.	GUI Exceptions
`AlertsPanel`, **`AlertTableModel`**	Alerts tab: Sort (Comparator), Resolve, Purge, Delete.	GUI JDBC Collections
`ChangesPanel`, **`RulesPanel`**, **`BroadcastLogPanel`** + their table models	The other 3 tabs: every recorded change, the 10 rules, and the send history.	GUI JDBC
`DbTask`	Runs database work in the background so the window never freezes (SwingWorker).	Threads Exceptions
`Renderers`	Coloured flags: NEW blue, SENT orange, RESOLVED green.	GUI Inheritance

employee — the business app (billing counter)

File(s)	What it does (plain words)	Rubric
`EmployeeMain`, **`EmployeeFrame`**	Product window: search, sort by column, Add / Save / Delete, error dialogs.	GUI JDBC Exceptions
`ProductTableModel`, **`EmployeeTask`**	Table data for the window; background database work.	GUI Threads

other folders

File(s)	What it does (plain words)	Rubric
`sql/setup.sql`	Creates the database, 5 tables, 20 sample products and the 3 triggers (the CCTV).	JDBC
`sql/reset-demo.sql`	Puts the demo data back to the start.	JDBC
`test/` (4 checks)	Self-tests: Phase12Check, DaoCheck, RuleCheck, DetectionCheck = 41 / 24 / 26 / 19 checks.	JDBC Inheritance

4. Each rubric line

Design Design

In simple words: Like a hospital with departments: each package does one job.

Where: 3 diagrams in `docs/diagrams`. Packages: common, rules, db, server, client, admin, employee.

Say: "Screens talk to the database only through the DAO classes. We used a Factory for alerts, read-only alert objects, and a queue for each dashboard."

If sir asks "Why no foreign keys?" → On purpose: a deleted product's history and a purged alert's log must stay. See the ER diagram.

Inheritance Inheritance & polymorphism

In simple words: One ID-card template, many card types. The guard just says "show your ID": each card answers for itself.

Where: **`Alert`** → 3 kinds; **`Rule`** → 7 kinds; used in **`AlertCard`** and **`RuleSet`**.

Say: "Alert is abstract; each child writes its own icon, colour and message. The dashboard calls the same method on any alert, and Java runs the right one at runtime."

If sir asks "Where's your own interface?" → We implement Java's: Runnable, Serializable, AutoCloseable, Comparator. Our shared logic needed an abstract class, because it has fields and a constructor.

Collections Collections

In simple words: Phone contacts (HashMap), bank token queue (BlockingQueue), guest list (HashSet).

Where: **`DashboardFrame`** (HashMap, EnumMap), **`ClientHandler`** (LinkedBlockingQueue), **`BroadcastServer`** (CopyOnWriteArrayList), **`AlertTableModel`** (sort + Comparator), **`ChangeDAO`** (LinkedHashMap).

Say: "A HashMap by alert id means a resolved alert updates its card instead of adding a duplicate. Each dashboard has its own queue."

If sir asks "Why CopyOnWriteArrayList?" → Three threads use the dashboard list at once; readers get a safe snapshot.

Exceptions Exception handling

In simple words: Plan B. An ATM out of cash shows a message, it doesn't explode.

Where: **`InvalidAlertException`**, **`InvalidProductException`** (our own, checked); rollback in **`ChangeDAO`**; reconnect in **`AlertListenerThread`**; dialogs in **`EmployeeFrame`**.

Say: "Our exceptions are checked so the caller must handle them. Try-with-resources closes everything. If MySQL or the server goes down, nothing crashes; it retries."

If sir asks "Checked or unchecked?" → Checked for problems we can handle (bad input, database down); unchecked only for bugs.

JDBC JDBC CRUD (2 marks)

In simple words: JDBC is the phone line to MySQL; the DAO is the one receptionist who makes every call.

Where: **`ProductDAO`** (full CRUD), **`AlertDAO`** (read / update / delete), **`ChangeDAO`** (JOIN + transaction), **`ManagedConnection`**, `setup.sql` (triggers).

Say: "Every statement is a PreparedStatement, which prevents SQL injection. Creating an alert is one transaction: all or nothing, like a UPI payment."

If sir asks "Show CRUD" → Employee app: Add, view, Save, Delete a product. Admin: Resolve (update), Purge (delete).

GUI GUI (Swing)

In simple words: Like a car dashboard: shows what's happening; the engine work happens elsewhere.

Where: **`AdminFrame`** (tabs, tables, Timer), **`EmployeeFrame`** (form, search, sort), **`DashboardFrame`** + **`AlertCard`** (live cards).

Say: "Three Swing apps. Tables use our own table models; coloured flags use a custom renderer."

If sir asks "Why doesn't the window freeze?" → Database work runs in a SwingWorker (**`DbTask`**), not on the screen thread.

Threads Threads

In simple words: Kitchen workers: one takes orders, one cooks, one delivers, all at once. Only the cashier (EDT) touches the cash drawer (the screen).

Where: **`ChangeDetectorThread`**, **`PollerThread`**, **`BroadcastServer`** (thread pool), **`ClientHandler`**, **`AlertListenerThread`**, **`DbTask`**. Show them live in **jconsole → Threads**.

Say: "We stop threads safely with a volatile flag plus interrupt(), never stop(). Database methods are synchronized because a connection can't be shared."

If sir asks "What if a dashboard is slow?" → It has its own queue and thread, so it can't slow the others.

Sockets Sockets (supports Design + Threads)

In simple words: A phone call: the server waits on number 5050; dashboards dial in and listen.

Where: **`BroadcastServer`** (ServerSocket), **`ClientHandler`** (ObjectOutputStream), **`AlertListenerThread`** (Socket, ObjectInputStream).

Say: "Alerts are Serializable objects sent over TCP. If the line drops, the dashboard redials every 3 seconds and keeps its cards."

If sir asks "What if the server crashes mid-send?" → The alert is sent again after restart; the dashboard sees the same id and updates the same card.

5. Demo order and who says what

Before sir arrives: MySQL running → `reset-demo.sql` → `build.cmd` → `run checks` (keep the result on screen) → diagrams open → code files open in tabs → all programs closed. **Only 5 minutes?** Do steps 1, 3, 4, 7, 9, 11, 13.

> **DON'T GET CAUGHT OUT**
>
> Run `reset-demo.sql` before every practice. Stock alerts fire only when stock *crosses* the line, so a second burst shows nothing new.

#	Who	Do this	Sir sees	Rubric
1	Anish	Architecture diagram	The whole system	Design
2	Anish / M5	UML + ER diagrams	Class families; 5 tables	Design Inheritance
3	Rajvi	`run client` first	"Reconnecting"	Exceptions Sockets
4	Anish	`run server`	Dashboard turns green	Threads Sockets
5	Anish	`jconsole` → Threads	The named threads	Threads
6	Vaishnavi / M4	`run admin`, `run employee`	4 tabs; 20 products	GUI JDBC
7	M5 → Anish	Workbench: price of id 3 = 49	CRITICAL card; Changes tab shows who + old/new	JDBC Inheritance
8	M4	Employee: coffee price 500, Save	CRITICAL *by alertapp*	JDBC GUI
9	M4	Price −5, Save	Error dialog, nothing saved	Exceptions
10	M4	Add a product, then Delete it	[ADD] LOW, then [DEL] HIGH	JDBC
11	Vaishnavi + Rajvi	Sort → Resolve → Purge → Broadcast Log	Same card turns grey; history kept	Collections JDBC
12	Anish	Workbench: stock = 0 for all	About 20 cards, smooth	Threads Collections
13	Rajvi / Anish	Ctrl+C server, edit, restart	Cards kept; reconnects; edit appears	Exceptions Threads
14	M5	`run checks` result + limits	41 / 24 / 26 / 19 passed	Testing

6. Quick questions

Q. Is it real-time?

Near real-time: 2–4 s. Two 2-second rounds (detect, then send). Sending is a true push; the dashboard never asks.

Q. Why triggers?

They run inside MySQL on every change from any tool, and save who, old value and new value.

Q. Why is Alert abstract?

An alert must be a specific kind. Shared data and checks live in the parent; each child writes its own display.

Q. Where is polymorphism?

`AlertCard` calls getIconText() on any Alert; the child decides. Same in **`RuleSet`** for rules.

Q. Why PreparedStatement?

Values are sent separately from the SQL, so no SQL injection.

Q. Where is the transaction?

`ChangeDAO`: save the alert + mark the change as done, both or neither.

Q. Why synchronized?

One database connection can't be used by two threads at once, like one ATM.

Q. How do you stop a thread?

A volatile flag set to false, plus interrupt() to wake it. join() waits for it.

Q. What if MySQL stops?

Warn once, retry every 2 s, reconnect automatically. No crash, no restart.

Q. What if the server stops?

Dashboards keep their cards, say Reconnecting, and reconnect. Missed changes are processed after restart.

Q. No duplicate cards: how?

HashMap by alert id: same id = same card, just updated.

Q. Why can Workbench save −10 but the app can't?

The app validates; the database doesn't. That's exactly why monitoring exists, and rule R1 catches it.

Q. Who made the change?

MySQL USER(): root = Workbench, alertapp = our app. Limit: all employees share one login.

Q. How did you test?

`run checks`: 4 programs, 110 checks, plus the full live run.

Honest limits (say them before sir finds them)

- 2–4 s delay (polling every 2 s; the interval is a setting).
- All employees share one database login, so we can't tell which employee made an edit.
- Only the products table is watched; the rules are written in Java.
- A dashboard opened later doesn't see older alerts (the Admin shows the full history).
- No login or encryption on the network: fine for a lab demo, not for a real company.

> **CLOSING LINE**
>
> We kept the scope honest, and inside it the system is solid: no lost alerts, no duplicates, and it recovers by itself.